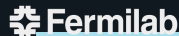

SC/DPS software: who owns what?

A clear, practical plan for the code we build together

Manuel Arroyave

DUNE slow-controls software discussion – 20 August 2026



First, agree on who is responsible

THE SIMPLE IDEA

Decide who owns the behavior. Then decide where the code lives.

A repository should answer three questions: who decides, who reviews, and what proves the release is ready.

01 What must it do?

Subsystem teams define the behavior and its limits.

02 Who makes it usable?

Slow Controls builds the operator bridge; DAQ and Computing help integrate it.

03 How do we trust it?

Interface, deployment, and safety owners review the release.

Start with the agreement, the installation, and DUNE practice

THE AGREEMENT

What should the system do?

The common project and subsystem specifications tell us which states, commands, and limits have been agreed.

Source: [Common SC project](#)

Source: [Example: PDS slow-controls specification](#)

THE INSTALLATION

What is really there?

Protection documents, rack maps, and HWDB tell us what is wired, named, and allowed.

[Protection system](#) [HWDB project](#)

THE DUNE WAY

How should the code fit?

DUNE repositories and Computing guidance show us how to structure, build, and hand over the code.

[DUNE-DAQ GitHub](#) [Computing basics](#)

START WITH PEOPLE

Draw the lines before naming repositories

When responsibility is clear, the Git structure becomes much easier to choose.

The work falls into three clear parts

THE DETECTOR SIDE

Electronics and detector services

For DAPHNE, BDE/WIB, and timing

We need agreed states and commands that both DAQ and Slow Controls understand.

For HVS and calibration

We need safe modes, trustworthy readback, and a record of every action.

THE INSTALLED SYSTEM

Power and rack services

PDU, sensor, outlet, and host names must match what is actually installed.

THE LINE WE DO NOT BLUR

Protection stays with its owner

DPS owns protection. DAQ and Computing stay involved where their systems meet ours.

Slow Controls connects the hardware to the operator

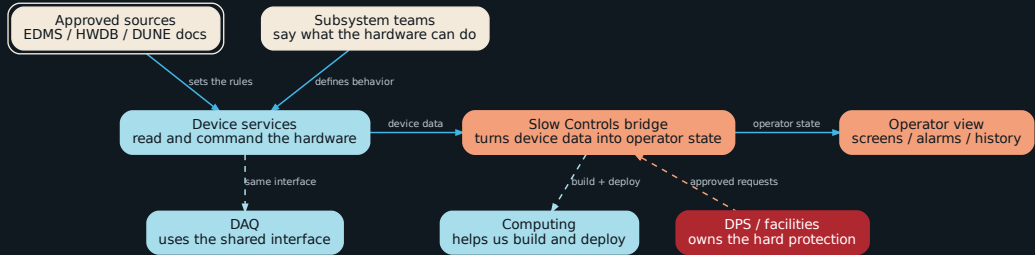


Diagram. The subsystem defines what the device can do. Slow Controls turns that into a reliable operator view. DPS still owns protection.

DAPHNE and WIB need one shared language

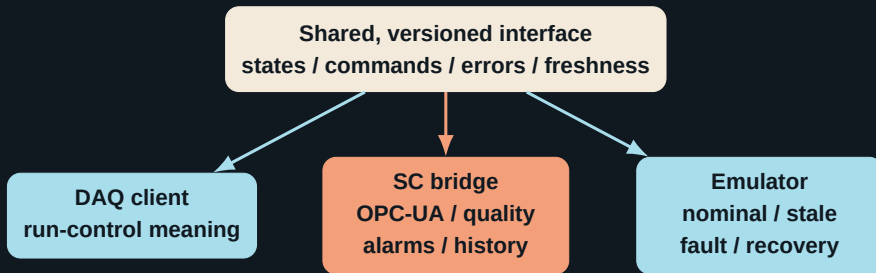


Diagram. DAQ, Slow Controls, and the emulator should all read the same interface—no private translations.

BEFORE WE RELEASE

All three must agree

The DAQ client, SC bridge, and emulator must prove that they understand the same version.

Rack software starts with what is really installed

FOLLOW THE FACTS

The operator view starts with real rack and outlet names.

Slow Controls can use those facts without taking ownership of the electrical system or its protection.

01 • INSTALLATION

Tell us what is there

Installation owners provide the rack, outlet, cable, and host mappings.

02 • SLOW CONTROLS

Turn facts into a useful view

The bridge adds data quality, alarms, command checks, and history.

03 • COMPUTING

Help us run it reliably

Computing reviews the hosts, network, build pipeline, artifacts, and deployment.

Seeing a problem is not the same as stopping it

SLOW CONTROLS CAN

Show and request

Make the situation visible

Show protection state, alarms, trends, and diagnostics.

Ask for an allowed action

Use an approved path with clear permissions and clear failure behavior.

DPS / FACILITIES MUST

Protect and authorize

Keep the hard protection independent

Interlocks must still work when ordinary software fails.

Approve any change to that line

Only the protection owner can change who carries this responsibility.

THE HARD RULE

The screen is not the safety system

Unless the protection owner explicitly approves it, an HMI, bridge, or ordinary service is only a monitor—not an interlock.

For detector electronics, SC owns the bridge—not the device

System	Slow Controls is responsible for	We work with
PDS / DAPHNE	The bridge and operator view OPC-UA mapping, data quality, alarms, and history.	PDS and DAQ Board behavior, the shared interface, and DAQ compatibility.
TPC / BDE / WIB	The BDE/WIB bridge Readiness, operator commands, and bridge behavior.	TPC/BDE and DAQ WIB limits and run-control integration.
Timing / TDE	The operator view State, quality, commands, alarms, and history.	Timing and DAQ Timing behavior and data-taking limits.

For services, SC owns the view—not the infrastructure

System	Slow Controls is responsible for	We work with
PDU / rack services	The monitoring bridge SNMP/IPMI mapping, stale-data rules, and alarms.	Electrical, rack, and Computing Installed facts, hosts, network, and deployment.
HVS / calibration	The operator view Methods, readback quality, alarms, and history.	Subsystem and safety owners Safe ranges, permissions, and approved modes.
DPS / facilities	Visibility Display, archive, alarms, and documented requests.	DPS / facilities The protection logic and hard interlocks.

NOW MAP IT TO GIT

Let the repository structure tell the same story

Someone opening the code should be able to see where the facts, interfaces, bridge, operator view, and tests belong.

The Git structure should match the way we work

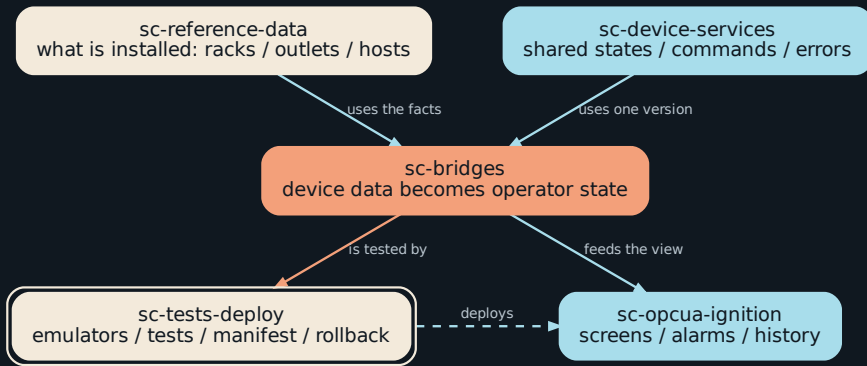


Diagram. The installed facts and shared interface feed the SC bridge. The operator view and release proof stay visible as separate jobs.

A reviewer should find four things in every repository

01

The source

Link the EDMS, HWDB, DUNE, or GitHub material that the code follows.

02

The people

Name the hardware, SC, DAQ, safety, and deployment reviewers who matter here.

03

The interface

Version the shared language. Say who uses it and what happens when it fails.

04

The proof

Include tests, compatibility results, a manifest, and a short rollback plan.

The README explains the promise; the release tag shows we kept it.

Start small and prove the whole path

Use DAPHNE or WIB; make one shared interface work three ways

01

Write the shared language

Define the states, commands, errors, data age, and compatibility rules.

02

Use it in DAQ and SC

One DAQ client and one Slow Controls bridge use exactly the same version.

03

Test the bad days too

An emulator checks normal operation, stale data, faults, and recovery.

Do not argue about names first. A working example will show us where the clean split belongs.

What we need to decide on Thursday

WHAT I AM ASKING FOR

Leave with clear owners and a first test

Choose the responsibility and review rules now. Let repository names follow from them.

1. Agree to assign responsibility before repositories.
2. Confirm that SC owns the DAPHNE/WIB operator bridges.
3. Choose where these repositories will live on GitHub.
4. Name the reviewers and choose DAPHNE or WIB for the first working example.

WE ARE DONE WHEN

One tagged release proves the idea

One shared interface works in a DAQ client, an SC bridge, and an emulator—with named reviewers.